```
a, b, c, d, e = *(1..2), 3, *[4, 5]    # a=1, b=2, c=3, d=4, e=5
```

Exactly one lvalue may be a splat. This makes it greedy—it will end up being an array, and that array will contain as many of the corresponding rvalues as possible. So, if the splat is the last lvalue, it will soak up any rvalues that are left after assigning to previous lvalues:

```
a, *b = 1, 2, 3    # a=1, b=[2, 3]
a, *b = 1          # a=1, b=[]
```

If the splat is not the last lvalue, then Ruby ensures that the lvalues that follow it will all receive values from rvalues at the end of the right side of the assignment—the splat lvalue will soak up only enough rvalues to leave one for each of the remaining lvalues.

```
*a, b = 1, 2, 3, 4          # a=[1, 2, 3], b=4
c, *d, e = 1, 2, 3, 4       # c=1, d=[2, 3], e=4
f, *g, h, i, j = 1, 2, 3, 4 # f=1, g=[], h=2, i=3, j=4
```

As with method parameters, you can use a raw asterisk to ignore some rvalues:

```
first, *, last = 1,2,3,4,5,6    # first=1, last=6
```

Nested Assignments

The left side of an assignment may contain a parenthesized list of terms. Ruby treats these terms as if they were a nested assignment statement. It extracts the corresponding rvalue, assigning it to the parenthesized terms, before continuing with the higher-level assignment.

```
a, (b, c), d = 1,2,3,4        # a=1, b=2, c=nil, d=3
a, (b, c), d = [1,2,3,4]      # a=1, b=2, c=nil, d=3
a, (b, c), d = 1,[2,3],4      # a=1, b=2, c=3, d=4
a, (b, c), d = 1,[2,3,4],5    # a=1, b=2, c=3, d=5
a, (b,*c), d = 1,[2,3,4],5    # a=1, b=2, c=[3, 4], d=5
```

Operator Plus Assignment

Ruby has a syntactic shortcut for the case where you want to apply an operation to a value and immediately assign that value the new result: `a = a + 2` may be written as `a += 2`. The second form is converted internally to the first. This means that operators you have defined as methods in your own classes work as you'd expect:

```
class List
  def initialize(*values)
    @list = values
  end

  def +(other)
    @list.push(other)
  end

end

a = List.new(1, 2)      # => [1, 2]
a += 3                  # => [1, 2, 3]
```